EXPERIMENT: 09

Name:- Abhishek Sharma **UID:-**23BCS11614
Branch:- BE-CSE **Section:-** KRG 2-B
Semester:- 6th **Date :-** 20-04-2026
Subject Name:- System Design **Subject Code:-** 23CSH-314

1.Aim:-

To design a high-level architecture for a scalable **Ticket Booking System** (similar to BookMyShow, Paytm, and Ticketmaster) that supports event discovery, real-time seat availability, and secure ticket booking with high availability and strong consistency.

2.Objectives:-

The objectives of this experiment are:

1. To understand the architecture of a large-scale ticket booking system.
2. To design APIs for event discovery, seat selection, and ticket booking.
3. To analyze functional and non-functional requirements of a booking platform.
4. To study real-time seat availability and concurrency handling.
5. To ensure strong consistency during ticket booking transactions.
6. To understand how event management and booking systems operate.

3.Procedure:-

1. Studied real-world platforms such as BookMyShow and Ticketmaster.
2. Identified key entities including Users, Events, Shows, Seats, Bookings, and Payments.
3. Analyzed workflow:
   User → Search Event → View Details → Select Seats → Payment → Booking Confirmation
4. Gathered functional requirements like search, booking, and seat tracking.
5. Analyzed non-functional requirements such as scalability, high availability, and consistency.
6. Designed RESTful APIs for event search, booking, and user management.
7. Studied caching mechanisms (Redis) for fast event search.
8. Understood concurrency handling using seat locking mechanisms.
9. Evaluated trade-offs:
   Availability (search)
   Consistency (booking)

4. Functional Requirements:-

i.Users should be able to register and log in. ii.Users should be able to search events by:

- Event Title
- City/Location
- Date

iii.Users should be able to view event details:

- Description
- Cast
- Duration
- Ratings iv.System should display:
- Available seats
- Booked seats

v.Users should be able to:

- Select seats
- Book tickets vi.System should:
- Lock seats temporarily during booking
- Prevent double booking vii.Users should be able to:
- View booking history
- Download tickets

5.Non Functional Requirements:-

i.The system must support **100 Million Daily Active Users (DAU)**.
ii.Search operations must have **high availability**. iii.Booking must ensure **strong consistency**.
iv.System latency:

- Search < 200 ms
- Booking < 2 seconds

v.System should be:

- Fault tolerant
- Highly scalable (microservices architecture) vi.Use caching (Redis) to improve performance. vii.Ensure secure transactions using HTTPS and authentication.

6.Core Entities:-

- User
- Event
- Venue
- Show

- Seat
- Booking
- Payment

7.API Design:-

i.User APIs:

- POST /auth/signup
- POST /auth/login **ii.Event APIs:**
- GET /api/v1/events
- GET /api/v1/events/{event_id} **iii.Search APIs:**
- GET /api/v1/events?city={city}&date={date}&title={title}

iv.Seat APIs:

- GET /api/v1/shows/{show_id}/seats

v.Booking APIs:

- POST /api/v1/bookings/lock
- POST /api/v1/bookings/confirm
- GET /api/v1/bookings

vi.Payment APIs:

- POST /api/v1/payments

8.Database Schema Design:-

i. Users Table

- user_id (PK)
- name
- email

ii. Events Table

- event_id (PK)
- title
- description
- language
- rating

iii. Venues Table

- venue_id (PK)
- name
- city

iv. Shows Table

- show_id (PK)
- event_id (FK)
- venue_id (FK)
- datetime

v. Seats Table

- seat_id (PK)
- show_id (FK)
- status (available, locked, booked)

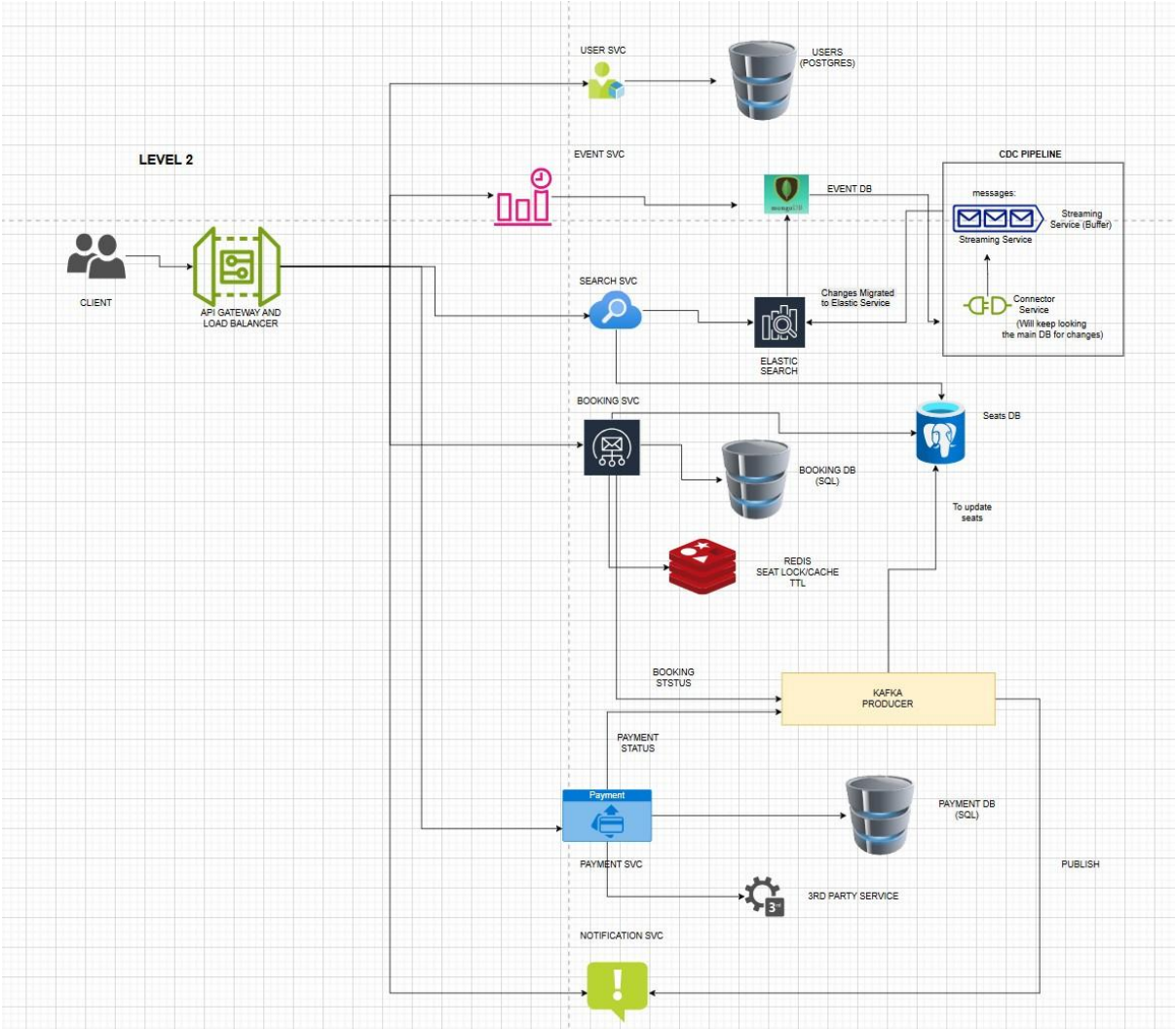
vi. Bookings Table

- booking_id (PK)
- user_id (FK)
- show_id (FK)
- status

vii. Payments Table

- payment_id (PK)
- booking_id (FK)
- status

9.HLD:-


LEVEL 2
CLIENT
API GATEWAY AND LOAD BALANCER
USER SVC
USERS (POSTGRES)
EVENT SVC
EVENT DB
CDC PIPELINE
messages:
Streaming Service (Buffer)
Streaming Service
Connector Service
(Will keep looking the main DB for changes)
SEARCH SVC
Changes Migrated to Elastic Service
ELASTIC SEARCH
BOOKING SVC
Seats DB
BOOKING DB (SQL)
To update seats
REDIS SEAT LOCK/CACHE TTL
BOOKING STSTUS
KAFKA PRODUCER
PAYMENT STATUS
Payment
PAYMENT DB (SQL)
PAYMENT SVC
PUBLISH
3RD PARTY SERVICE
NOTIFICATION SVC